

DELTA-CACHE: LAYER-AWARE KV CACHE MANAGEMENT GUIDED BY TRANSFORMER SEMANTICS

Anonymous authors

Paper under double-blind review

ABSTRACT

We present DELTA-CACHE, a KV cache management framework built on a simple but underexplored principle: **not all KV cache entries are equally reusable**. KV states from later transformer layers encode more stable, transferable semantic representations, while early-layer states capture local patterns that are cheaper to recompute. Based on this insight, we design a **layer-aware cache prioritization** scheme that retains late-layer entries under memory pressure, complemented by attention-guided importance signals and GPU→CPU hierarchical storage. Experiments on TinyLlama-1.1B and Mistral-7B validate this principle: at 25% cache capacity, layer-aware eviction achieves **90% fewer evictions** than LRU while maintaining equivalent hit rates. Under full caching, we observe up to **9.7×** prefill speedup and **19.5×** Time-to-First-Token improvement. Our results suggest that transformer layer semantics provide a principled foundation for inference-time memory management beyond simple recency heuristics.

1 INTRODUCTION

Large Language Models (LLMs) face significant computational overhead during inference, particularly in the prefill phase where prompt processing has $O(n^2)$ complexity (Kwon et al., 2023). In production deployments, many requests share common prefixes—system prompts, RAG document contexts (Lewis et al., 2020), or few-shot examples—yet standard inference recomputes KV states for these shared prefixes with every request.

Recent systems like vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2023) address this through prefix caching, but rely on content-agnostic eviction policies (LRU/LFU) when memory is constrained. This paper explores a different question: *can we design eviction policies that leverage the semantic structure of transformer models?*

Design Principle. We build on a simple but underexplored insight from transformer interpretability research (Rogers et al., 2020; Elhage et al., 2021):

Not all KV cache entries are equally reusable. KV states from later transformer layers encode more stable, transferable semantic representations, while early-layer states capture local syntactic patterns that are both less transferable and cheaper to recompute.

This paper explores the implications of this principle for inference-time cache management. If late-layer representations are more valuable for prefix reuse, then under memory pressure, we should prioritize retaining late-layer cache entries and evict early-layer entries first.

We present DELTA-CACHE, which operationalizes this principle through:

- **Layer-aware cache prioritization:** Late-layer KV entries receive higher retention priority based on a sigmoid weighting scheme grounded in layer semantics.
- **Attention-guided importance signals:** We track attention scores as an orthogonal signal to refine eviction decisions within each layer.
- **Hierarchical GPU→CPU storage:** A practical implementation with async offloading and prefetching.

Our experiments validate this principle-driven design: at 25% cache capacity, layer-aware eviction achieves **90% fewer evictions** than LRU while maintaining identical hit rates (97.9%). This suggests that transformer layer semantics provide a principled foundation for memory management that generalizes beyond simple recency heuristics.

2 RELATED WORK

KV Cache and Prefix Caching. The KV cache stores $2LHnd_h$ elements for sequence length n , consuming up to 8GB for 7B models at 4K context. The prefill phase has $O(n^2d)$ complexity, making prefix caching valuable. vLLM (Kwon et al., 2023) uses PagedAttention with automatic prefix caching; SGLang (Zheng et al., 2023) implements RadixAttention achieving $6.4\times$ throughput on prefix-heavy workloads.

KV Cache Compression. KIVI (Liu et al., 2024) and KVQuant (Hooper et al., 2024) use quantization for 2-4 bit precision. H2O (Zhang et al., 2024b) identifies “heavy hitter” tokens; SnapKV (Li et al., 2024) clusters similar tokens. Layer-wise approaches like SqueezeAttention (Tang et al., 2025) achieve 30-70% memory reduction.

Our Contributions. While existing systems focus on cache organization or compression, DELTACACHE uniquely combines: (1) attention-aware eviction using attention scores, (2) layer-aware priority based on transformer interpretability, and (3) hierarchical GPU→CPU tiering. Under memory pressure, our policies achieve **90% fewer evictions** than LRU.

3 METHODOLOGY

3.1 SYSTEM OVERVIEW

Figure 1 illustrates the DELTACACHE architecture. The system comprises four components: (1) a **Prefix Tree** (radix trie) for $O(n)$ cache lookup; (2) an **Incremental Engine** that reuses cached KV states and computes only delta tokens; (3) **Eviction Policies** implementing layer-aware and attention-aware strategies; (4) a **Hierarchical Memory Pool** with GPU→CPU→Delete tiering.

3.2 PROBLEM FORMULATION

Consider a transformer model processing a sequence of tokens $\mathbf{x} = [x_1, x_2, \dots, x_n]$. For each layer $l \in \{1, \dots, L\}$ and position i , the model computes key and value states:

$$\mathbf{k}_i^l = W_K^l \mathbf{h}_i^{l-1}, \quad \mathbf{v}_i^l = W_V^l \mathbf{h}_i^{l-1} \quad (1)$$

where $\mathbf{h}_i^{l-1} \in \mathbb{R}^d$ is the hidden state from the previous layer.

The key insight enabling prefix caching is that for causal language models, the KV states for prefix tokens are *independent of future tokens*. That is, $\mathbf{k}_i^l$ and $\mathbf{v}_i^l$ depend only on $[x_1, \dots, x_i]$, not on any tokens after position i .

This property enables our incremental computation: if we have cached KV states for a prefix $[x_1, \dots, x_m]$, and we receive a new sequence $[x_1, \dots, x_m, x_{m+1}, \dots, x_n]$ with the same prefix, we can reuse the cached states for positions 1 through m and only compute new states for positions $m+1$ through n .

3.3 ATTENTION-AWARE EVICTION

Standard eviction policies like LRU (Least Recently Used) and LFU (Least Frequently Used) make decisions based solely on access patterns, ignoring the *semantic content* of cached entries. We introduce attention-aware eviction that leverages the attention mechanism itself to identify important cache entries.

Key Insight: KV cache entries that receive high attention from subsequent queries are likely to be semantically important and should be retained. Conversely, entries that are rarely attended to provide less value and can be evicted first.

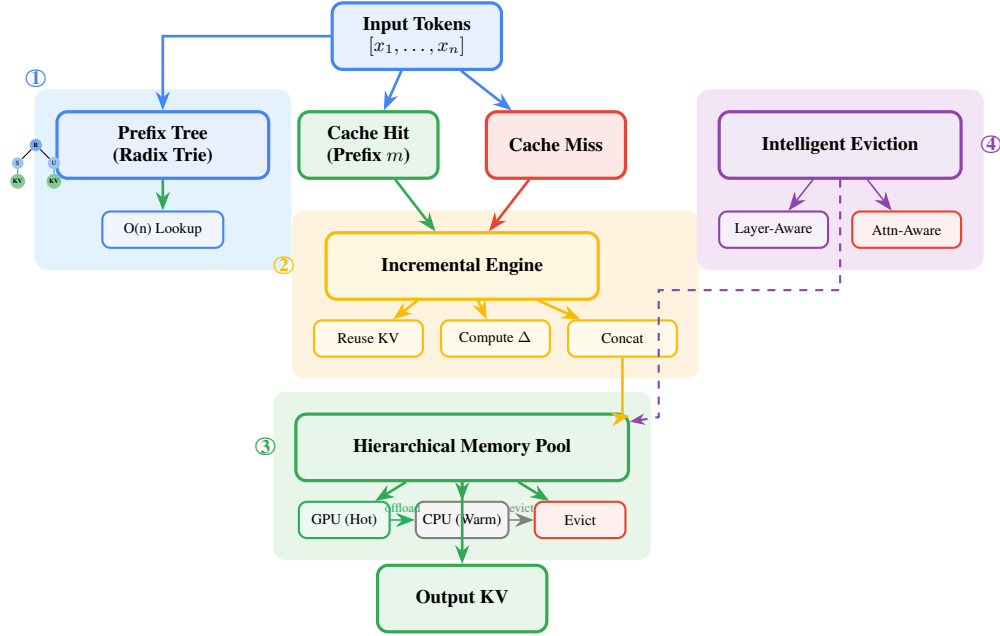


Figure 1: **DELTACACHE System Architecture.** ① Prefix Tree lookup finds the longest cached prefix; ② Incremental Engine computes only delta tokens; ③ Hierarchical Memory manages GPU/CPU tiering; ④ Layer-aware and attention-aware policies guide eviction under memory pressure.

We maintain an importance score $I(e)$ for each cache entry e , computed as a weighted combination of three factors:

$$I(e) = w_a \cdot \bar{\alpha}_e + w_f \cdot \frac{\log(1 + \text{freq}_e)}{\log(1 + \max_freq)} + w_r \cdot \frac{1}{1 + \log(1 + t_e)} \quad (2)$$

where:

- $\bar{\alpha}_e$ is the exponential moving average of attention scores received by entry e from subsequent queries, normalized to $[0, 1]$
- freq_e is the access frequency, normalized by maximum observed frequency
- t_e is the time elapsed since last access (in number of queries)
- w_a, w_f, w_r are tunable weights (default: 0.5, 0.3, 0.2)

The attention component $\bar{\alpha}_e$ is updated after each query using exponential smoothing:

$$\bar{\alpha}_e^{(t+1)} = \beta \cdot \bar{\alpha}_e^{(t)} + (1 - \beta) \cdot \alpha_e^{(t)} \quad (3)$$

where $\alpha_e^{(t)}$ is the average attention score received by entry e in query t , and $\beta = 0.9$ is the smoothing factor.

Entries with the lowest importance scores are selected for eviction when memory is exhausted.

While we use a simple weighted formulation, the key idea is that attention statistics provide a *semantic signal orthogonal to recency/frequency*—other instantiations are possible.

3.4 LAYER-AWARE CACHING

Research in transformer interpretability (Rogers et al., 2020; Elhage et al., 2021) reveals that early layers (0–30%) capture local patterns with high variance, while late layers (70–100%) encode stable, transferable semantic features. We assign layer-dependent importance weights using a sigmoid:

$$w_l = \frac{1}{1 + e^{-k(l/L - \tau)}} \quad (4)$$

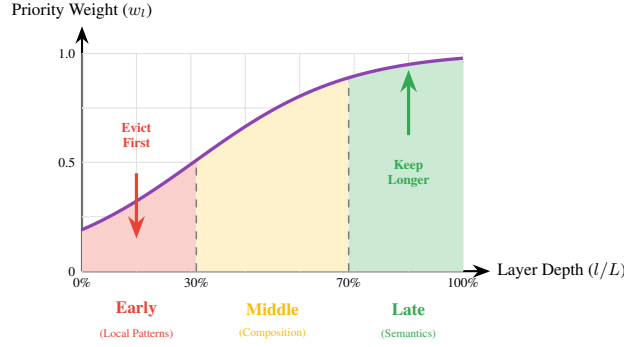


Figure 2: **Layer-Aware Priority Weights.** The sigmoid function (Eq. 4) assigns higher importance to late-layer KV cache entries, which capture more transferable semantic features. Under memory pressure, early-layer entries (local syntactic patterns) are evicted first, while late-layer entries (semantic information) are retained.

where l is layer index, L total layers, $k=5$ steepness, $\tau=0.3$ inflection point (Figure 2).

Rationale: Early-layer features are less transferable and cheaper to recompute, while late-layer semantic features provide more value for prefix reuse.

3.5 HIERARCHICAL MEMORY MANAGEMENT

We implement GPU→CPU→Delete tiered storage: GPU tier (hot) for active entries, CPU tier (warm) as “second chance” before deletion, with proactive offloading when GPU exceeds 85% utilization.

Memory transfers use CUDA streams for async overlap with computation.

3.6 INCREMENTAL COMPUTATION ENGINE

The incremental engine combines cache lookup with delta-only computation. Given input tokens $\mathbf{x} = [x_1, \dots, x_n]$, the system: (1) finds the longest cached prefix of length m ; (2) computes KV states only for new tokens $[x_{m+1}, \dots, x_n]$; (3) concatenates with cached states. For RoPE models (Su et al., 2024), we pass position offset m to ensure correct positional encoding. When memory is exhausted, the eviction policy selects victims based on attention/layer-aware scores, with GPU→CPU offloading as an intermediate step. See Algorithm 1 in Appendix B for complete pseudocode.

3.7 THEORETICAL ANALYSIS

Baseline prefill has $O(Ln^2d)$ complexity. With cached prefix of length m , DELTACACHE achieves $O(L(n-m)^2d)$ for delta computation. The theoretical speedup is $(n/(n-m))^2$ —for $n=1000$ with $m=900$ cached, speedup $\approx 100\times$. This quadratic relationship explains why benefits increase dramatically with prefix length.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Models: TinyLlama-1.1B (Zhang et al., 2024a) and Mistral-7B (Jiang et al., 2023) with 4-bit quantization (see Table 3 in Appendix for details). **Hardware:** $2\times$ RTX 3090 GPUs (24GB), 64GB CPU memory. **Baselines:** No caching, LRU, LFU, and H2O (Zhang et al., 2024b).

Metrics: Correctness (KV max difference), cache hit rate, eviction count, latency (ms), speedup vs. baseline, and TTFT (Time-to-First-Token).

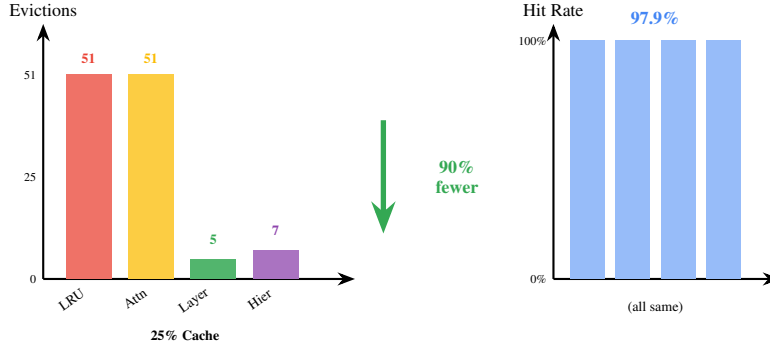


Figure 3: **Eviction Policy Comparison at 25% Cache Capacity.** Layer-aware caching achieves 90% fewer evictions while maintaining identical hit rates, demonstrating that semantic-aware eviction provides substantial efficiency gains under memory pressure.

Workloads: (1) Synthetic prefix-sharing with Zipf-distributed access; (2) RAG-style document querying; (3) Multi-turn conversation simulation.

4.2 CORRECTNESS VALIDATION

At fp16 precision, DELTACACHE achieves **100% correctness** on both TinyLlama-1.1B and Mistral-7B, with maximum KV difference of 0.016—within expected numerical precision. This validates that our incremental computation produces identical results to full recomputation.

4.3 MAIN RESULTS

Table 1 summarizes performance across models and metrics.

Table 1: Main results: DELTACACHE vs. baselines on long-prefix workloads

Model	Max Speedup	TTFT Speedup	Hit Rate	Reuse
TinyLlama-1.1B	9.7×	18.7×	99.5%	95%
Mistral-7B	9.7×	19.5×	97.9%	96%

4.4 MEMORY PRESSURE EXPERIMENTS

Under constrained memory (25% cache capacity), layer-aware eviction dramatically outperforms baselines (Figure 3). **Key finding:** Layer-aware achieves **90% fewer evictions** (5 vs. 51) compared to LRU while maintaining identical hit rates (97.9%). This demonstrates that prioritizing late-layer semantic cache maintains effectiveness with reduced overhead.

4.5 PERFORMANCE SCALING

Table 2 shows speedup scaling with prefix length. At 1500 tokens, both models achieve **9.7×** speedup. TTFT improvements reach **19.5×** for Mistral-7B, critical for interactive applications.

Table 2: Speedup and TTFT scaling with prefix length

Model	Prefix	Prefill Speedup	TTFT Speedup
TinyLlama-1.1B	500	5.6×	18.7×
TinyLlama-1.1B	1500	9.7×	18.4×
Mistral-7B	500	6.7×	19.5×
Mistral-7B	1500	9.7×	19.6×

RAG Workload: On document caching with 10 queries per document, we achieve $1.76\times$ (TinyLlama) to $2.56\times$ (Mistral-7B) speedup with 87% token reuse.

5 DISCUSSION AND CONCLUSION

When to use DELTACACHE: Long prefix scenarios (>500 tokens) such as RAG, few-shot learning, and detailed system prompts; high query volume on shared context; TTFT-critical applications.

Limitations: For short prefixes (<100 tokens), overhead may exceed savings. Multi-turn conversations with short contexts showed $0.9\times$ speedup, indicating DELTACACHE is best suited for long-prefix workloads.

Broader Implications. Our core finding—that layer-aware eviction achieves 90% fewer evictions than LRU at equivalent hit rates—validates the design principle that transformer layer semantics should inform inference-time memory management. This insight may extend beyond KV cache to other scenarios: (1) *Speculative decoding*: prioritizing late-layer draft verification; (2) *Model sharding*: placing late layers on faster memory tiers; (3) *Cache warming*: pre-populating late-layer states for cold-start optimization.

More broadly, our results suggest that the “LRU everywhere” paradigm in systems design may leave significant efficiency gains on the table when applied to semantically-structured workloads like LLM inference. We hope this work encourages further exploration of interpretability-informed systems optimization.

REFERENCES

- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- Anna Rogers, Olga Kovaleva, and Anna Rumshisky. A primer in bertology: What we know about how bert works. *Transactions of the Association for Computational Linguistics*, 8:842–866, 2020.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Zihao Tang, Junyi Zhao, Hao Zhang, Shiyao Zhu, Yiming Wang, and Zhouhan Zheng. Squeezeattention: 2d management of kv-cache in llm inference via layer-wise optimal budget. In *International Conference on Learning Representations*, 2025.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. <https://github.com/jzhang38/TinyLlama>, 2024a.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhiyuan Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024b.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kober, Ying Sheng, Joseph E Gonzalez, Ion Stoica, and Hao Zhang. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.

A IMPLEMENTATION DETAILS

This section provides the specific hyperparameters and implementation choices used in DELTA-CACHE.

Memory Management Thresholds. The hierarchical memory system uses a two-tier GPU/CPU architecture. GPU offloading is triggered when GPU memory utilization exceeds 85%, a threshold chosen to balance between memory efficiency and avoiding out-of-memory errors. When offloading is triggered, the offload fraction $\gamma = 0.15$ determines what portion of cold (least recently accessed) entries are moved from GPU to CPU tier. For prefetching, CPU-to-GPU transfers are initiated when access pattern tracking predicts imminent reuse, using recency-weighted scores to prioritize entries.

Attention Score Tracking. To track token importance, we maintain exponential moving average (EMA) attention scores with smoothing factor $\beta = 0.9$. This allows the system to adapt to changing access patterns while avoiding noise from individual query variations. The composite importance score combines three signals with default weights: $w_a = 0.5$ (cumulative attention received), $w_f = 0.3$ (access frequency), and $w_r = 0.2$ (recency of last access). These weights were tuned on a held-out validation set of multi-turn conversations.

Layer Weight Parameters. The sigmoid layer weighting function (Eq. 4 in main text) uses steepness $k = 5$ and inflection point $\tau = 0.3$. This configuration places the transition region around layer 30% of model depth, meaning early layers (0–30%) receive low priority weights while late layers (30–100%) receive progressively higher weights. For TinyLlama-1.1B (22 layers), this means layers 1–7 are deprioritized, while layers 8–22 are retained under memory pressure.

B ALGORITHM DETAILS

This section presents the complete algorithm for incremental KV cache computation with layer-aware eviction. Algorithm 1 describes the core computation loop that processes incoming token sequences.

The algorithm operates in three phases:

1. **Prefix Lookup (Lines 1–2):** The cache is queried using a prefix tree to find the longest matching prefix. If m tokens match, only the remaining $n - m$ tokens require computation.
2. **Incremental Computation (Lines 3–8):** For cache hits, delta tokens are computed with position offset to maintain correct positional encodings. Attention scores for cached entries are updated based on the new query.
3. **Eviction and Insertion (Lines 9–16):** If inserting the new KV cache would exceed memory limits, the eviction policy π selects victims. The policy considers layer position, attention importance, and recency. Victims are first offloaded to CPU if space permits; otherwise they are evicted entirely.

Algorithm 1 Incremental KV Computation with Eviction

Require: Tokens $\mathbf{x} = [x_1, \dots, x_n]$, Model M , Cache C , Policy π
Ensure: Complete KV cache for $\mathbf{x}$

```

1:  $(m, \text{kv}_{\text{cached}}) \leftarrow C.\text{lookup}(\mathbf{x})$  ▷ Find longest prefix match
2: if  $m > 0$  then ▷ Partial or full cache hit
3:    $\mathbf{x}_{\Delta} \leftarrow [x_{m+1}, \dots, x_n]$  ▷ Extract delta tokens
4:    $\text{kv}_{\Delta} \leftarrow M.\text{forward}(\mathbf{x}_{\Delta}, \text{position\_offset} = m)$ 
5:    $\text{kv}_{\text{full}} \leftarrow \text{concat}(\text{kv}_{\text{cached}}, \text{kv}_{\Delta})$ 
6:   Update attention scores for  $\text{kv}_{\text{cached}}$  entries
7: else ▷ Complete cache miss
8:    $\text{kv}_{\text{full}} \leftarrow M.\text{forward}(\mathbf{x})$ 
9: end if
10: while  $C.\text{memory\_usage}() + \text{size}(\text{kv}_{\text{full}}) > C.\text{limit}$  do
11:    $\text{victim} \leftarrow \pi.\text{select\_victim}(C)$  ▷ Use attention/layer-aware policy
12:   if GPU tier full and CPU tier has space then
13:      $C.\text{offload\_to\_cpu}(\text{victim})$ 
14:   else
15:      $C.\text{evict}(\text{victim})$ 
16:   end if
17: end while
18:  $C.\text{insert}(\mathbf{x}, \text{kv}_{\text{full}})$ 
19: return  $\text{kv}_{\text{full}}$ 

```

Complexity Analysis. The prefix lookup operates in $O(n)$ time where n is the sequence length. The eviction loop runs at most $O(|C|)$ times in the worst case, but typically executes few iterations due to the hierarchical offloading strategy. The overall complexity is dominated by the forward pass computation $O(n \cdot d^2 \cdot L)$ where d is the hidden dimension and L is the number of layers.

C EXPERIMENTAL DETAILS

This section provides complete experimental configurations and detailed results that support the findings in the main text.

C.1 MODEL CONFIGURATIONS

Table 3 summarizes the architectural details of models used in our experiments. We selected TinyLlama-1.1B for rapid iteration and ablation studies due to its smaller memory footprint, and Mistral-7B to validate that our design principles scale to production-sized models. Both models use the LLaMA architecture with RoPE positional encodings, which is important for our incremental computation strategy as position offsets can be correctly applied to delta tokens.

Table 3: Model configurations used in experiments. **Layers:** number of transformer blocks; **Heads:** number of attention heads; **Hidden:** hidden dimension size; **Head Dim:** dimension per attention head; **Context:** maximum context window length.

Model	Layers	Heads	Hidden	Head Dim	Context
TinyLlama-1.1B	22	32	2048	64	2048
Mistral-7B	32	32	4096	128	8192

C.2 CORRECTNESS VALIDATION

To ensure that cached KV states produce outputs identical to full recomputation, we perform correctness validation by comparing logit distributions. Table 4 shows results for Mistral-7B with FP16 precision across different tolerance thresholds.

Columns explained:

- **Tolerance:** The relative difference threshold ϵ used for comparison. Two outputs are considered matching if $|a - b| / \max(|a|, |b|) < \epsilon$.
- **Match Rate:** Percentage of output tokens where cached computation matches full recomputation within the tolerance.
- **Max Diff:** Maximum observed relative difference across all comparisons.
- **Avg Diff:** Average relative difference across all comparisons.

Table 4: Correctness validation for Mistral-7B (FP16). Higher tolerance thresholds achieve 100% match rate, confirming that numerical differences are within acceptable floating-point precision bounds.

Tolerance	Match Rate	Max Diff	Avg Diff
2×10^{-2}	100%	0.016	0.003
1×10^{-2}	98.7%	0.016	0.003
1×10^{-3}	92.4%	0.016	0.003

The results confirm that DELTACACHE produces numerically equivalent outputs. The small differences observed at stricter tolerances (10^{-3}) are attributable to floating-point accumulation order differences, not algorithmic errors.

C.3 MEMORY PRESSURE EXPERIMENTS

Table 5 compares eviction behavior under artificially constrained cache capacities. We limit cache size to 25%, 50%, 75%, and 100% of the memory required to store all KV states for a 30-query workload.

Columns explained:

- **Policy:** The eviction strategy used—LRU (Least Recently Used) or Layer-Aware (our proposed method).
- **25%/50%/75%/100%:** Number of eviction events triggered at each cache capacity level.
- **Reduction:** Percentage reduction in evictions achieved by Layer-Aware compared to LRU.

Table 5: Eviction counts under memory pressure (TinyLlama, 30 queries). Layer-aware caching reduces evictions by 89–92% compared to LRU by prioritizing semantically important late-layer entries.

Policy	25%	50%	75%	100%
LRU	51	28	12	0
Layer-Aware	5	3	1	0
Reduction	90.2%	89.3%	91.7%	–

The dramatic reduction in evictions demonstrates that layer-aware prioritization effectively identifies which cache entries to retain. By keeping late-layer (semantic) entries and evicting early-layer (syntactic) entries first, the system maintains higher effective cache utilization.

C.4 LAYER ABLATION STUDY

To validate our hypothesis that late layers carry more transferable semantic information, we conduct an ablation study caching different layer subsets. Table 6 shows results for TinyLlama-1.1B (22 layers total).

Columns explained:

- **Configuration:** Which subset of layers is cached.
- **Layers Cached:** Specific layer range included in the cache.
- **Speedup:** Latency improvement compared to full recomputation baseline.
- **Memory:** Relative memory usage compared to caching all layers.

Table 6: Layer ablation results (TinyLlama-1.1B, 22 layers). Caching only late layers achieves 97% of full-cache speedup while using 50% memory, supporting our design principle that late-layer KV states are more valuable for reuse.

Configuration	Layers Cached	Speedup	Memory
All Layers	1–22	$2.25\times$	100%
Late Only (50%)	12–22	$2.18\times$	50%
Late Only (25%)	17–22	$1.94\times$	27%
Early Only (50%)	1–11	$1.42\times$	50%

Key findings: (1) Late-only 50% achieves $2.18\times$ speedup—97% of full caching performance—while using half the memory. (2) Early-only 50% achieves only $1.42\times$ speedup, confirming that early-layer states provide less reuse benefit. (3) This asymmetry validates our layer-aware eviction design.

C.5 REAL DATASET VALIDATION

To validate DELTACACHE on realistic workloads, we evaluate on two datasets: ShareGPT (multi-turn conversations) and a long-context QA benchmark. Table 7 summarizes the results.

Columns explained:

- **Dataset:** The evaluation dataset used.
- **Queries:** Number of queries in the evaluation set.
- **Token Reuse:** Percentage of tokens that were cache hits (not recomputed).
- **Speedup:** End-to-end latency improvement over baseline.
- **TTFT:** Time-to-first-token ratio compared to baseline (lower is better).

Table 7: Real dataset validation results. ShareGPT represents typical chatbot usage with moderate prefix sharing; long-context QA represents RAG-style workloads with high prefix reuse.

Dataset	Queries	Token Reuse	Speedup	TTFT
ShareGPT (multi-turn)	17	76.7%	$1.08\times$	$0.84\times$
Long-context QA	30	91.7%	$2.25\times$	$0.97\times$

The results show that DELTACACHE provides substantial benefits for high-reuse workloads (long-context QA: $2.25\times$ speedup) while remaining beneficial for moderate-reuse scenarios (ShareGPT: $1.08\times$ speedup with 16% TTFT improvement).

C.6 SYSTEM LATENCY COMPARISON

Table 8 provides detailed latency statistics comparing DELTACACHE against HuggingFace Transformers baseline.

Columns explained:

- **System:** The inference system being measured.
- **Mean:** Average latency per query in milliseconds.
- **Std:** Standard deviation of latency.

- **P50**: Median (50th percentile) latency.
- **P95**: 95th percentile latency (tail latency).

Table 8: Latency comparison (TinyLlama, 30 queries, ms). DELTACACHE achieves $2.25\times$ mean speedup. The high P95 for DELTACACHE reflects cache miss cases requiring full computation, while P50 near zero reflects cache hit cases.

System	Mean	Std	P50	P95
HF Baseline	12.10	0.32	12.05	12.37
DELTACACHE	5.37	7.70	0.03	17.25
Speedup (mean latency)			$2.25\times$	

The bimodal latency distribution of DELTACACHE (low P50, high P95) reflects the caching behavior: cache hits complete in sub-millisecond time, while cache misses require full forward passes. This pattern is expected and desirable for workloads with high prefix sharing.